
University of Michigan–Ann Arbor

Department of Electrical Engineering and Computer Science

EECS 475 Introduction to Cryptography, Fall 2024

Instructor: Ke Wu

GSI: Forest Zhang

Homework 4

Student: Shikha Nukala (desigirl)

Collaborators (if any):

Please refer to the course information document for important collaboration policies. Among other policies, you must list all collaborators and, moreover, cite any external resources that you have used for each exercise. You must write all solutions on your own and may not share this material outside class.

Due: Nov 7, 2024 (at 11:59 PM US Eastern Time)

Exercise 1

(15 points) Prove that any public key encryption (even randomized) cannot be perfectly secure. (Hint: Consider a brute force attack.)

Solution

Information:

Perfect security means that knowing the ciphertext of a message under the chosen encryption scheme does not reveal any more information about the plaintext than what is already known. In other terms, $P(M = m | C = c) = P(M = m)$. Public key encryption is when we have a public key pk and a corresponding private key sk . Anyone can encrypt a message with pk but only the person who knows sk can decrypt an encrypted message to receive the plaintext. Since perfectly secure encryption means that the adversary cannot retrieve any more information about the plaintext from a given ciphertext regardless of whether they are unbounded or polynomially bounded, we can construct a brute force attack where the adversary uses pk to encrypt every message m in message space M and compares each outputted ciphertext to the sender's ciphertext. If the ciphertexts match, then the attacker knows the original message as the message they tried encrypting from the message space.

Formal Proof:

Assume that there exists a public key encryption (that is randomized) that is perfectly secure. That means there is a public key encryption whose ciphertexts do not reveal any more information about the plaintext to an unbounded adversary or a PPT adversary. Therefore, we would not be able to construct an adversary that can break perfect security of this encryption scheme. For this public key encryption, assume we have public key pk , private key sk , message space M , random generation algorithm Gen_r , encryption scheme Enc , and decryption scheme Dec . Assume there is an adversary A that tries to break the perfect security of this encryption.

The encryption scheme is defined as such (El Gamal encryption for a randomized encryption scheme):

$(G, q, g) \leftarrow Gen_r(1^n), x \leftarrow Z_q(\text{uniformrandom}), h = g^x$

$pk = (G, q, g, h), sk = (G, q, g, x)$

$Enc(pk, m) = c$ where $y \leftarrow Z_q(\text{uniformrandom}), m \in G, c = (m * h^y, g^y)$

$Dec(sk, c) = m$ where $m = \frac{c_1}{(c_2)^x}, c = (c_1, c_2)$

We can construct A such that:

A has access to pk and Enc

For every $m \in G$, A computes $Enc(pk, m)$ where $y \leftarrow Z_q(\text{uniformrandom})$.

For each possible m , A computes $|Z_q|$ ciphertexts using different possible y inputs and Enc and compares possibilities until A finds a ciphertext c_A that is equal to c . Then A knows message m_A is the original message.

Note here that for an unbounded adversary there is a almost 1 probability that the adversary correctly outputs the corresponding message to the ciphertext that the encryption scheme has output from the sender. Note there can be a negligible probability that the message space the adversary chooses does not contain the message of the sender.

Using this unbounded brute force adversary A , we have broken perfect security for a public key encryption that is randomized, therefore for any non-randomized and randomized public encryption scheme, it cannot be that the scheme is perfectly secure, breaking our assumption and proving the statement.

Exercise 2

(20 points) Let $n \in \mathbb{Z}$ be a positive integer. Consider the following set

$$\mathbb{G}_n := \{an + 1 \mid a \in \{0, \dots, n-1\}\}.$$

Every element of $\mathbb{G}_n$ also belongs to the multiplicative group $\mathbb{Z}_{n^2}^*$ (recall that $\mathbb{Z}_{n^2}^*$ contains all integers from $\{0, \dots, n^2 - 1\}$ that are coprime with n^2 .)

Show that $\mathbb{G}_n$ is a multiplicative subgroup of $\mathbb{Z}_{n^2}^*$. That is, for the multiplication module n^2 operation, the elements of G_n satisfy the following group axioms:

- *Closure*: For every $g_1, g_2 \in \mathbb{G}_n$, $g_1 \cdot g_2 \in \mathbb{G}_n$.

- *Identity:* There exists an element $e \in \mathbb{G}_n$ such that for every $g \in \mathbb{G}_n$, $e \cdot g = g \cdot e = g$.
- *Inverse:* For every $g \in \mathbb{G}_n$, there exists a $g^{-1} \in \mathbb{G}_n$ such that $g \cdot g^{-1} = g^{-1} \cdot g = e$.
- *Associativity:* For all $g, h, k \in \mathbb{G}_n$, $g \cdot (h \cdot k) = (g \cdot h) \cdot k$.

Solution

Closure:

For any $g_1, g_2 \in G$, there exist x and $y \in \{0, \dots, n-1\}$ such that $g_1 = xn + 1$ and $g_2 = yn + 1$. Then we would get that $g_1 * g_2 = (xn + 1)(yn + 1)$. Expanding this out we get $xyn^2 + xn + yn + 1$. Since we have coprimes of n^2 in the group, we can take the $\text{mod } n^2$ of the output $xyn^2 + xn + yn + 1$ to see if it is in the group G_n . Thus $xyn^2 + xn + yn + 1 \text{ mod } n^2$ becomes $xn + yn + 1$ due to the n^2 term. This becomes $(x+y)n+1 \text{ mod } n^2$ which is the form of $an+1$. We know $\mathbb{G}_n := \{an+1 \mid a \in \{0, \dots, n-1\}\}$ so $(x+y) \text{ mod } n \in \{0, \dots, n-1\}$. By multiplication properties we know if x and y are both coprime with n^2 , then $x * y$ is also coprime with n^2 because the factors remain the same. Therefore closure holds.

Identity:

If we set $a = 0 \in G_n$, we get that $e = 1$. Therefore by testing identity, we get that for every $g \in G_n$, $1 * g = g = g * 1 = g$. Therefore by setting $a = 0$ we get an element that satisfies the identity property of a subgroup.

Inverse:

To get an inverse for an element $g = an + 1$ where $a \in \{0, \dots, n-1\}$, we need to have another element $g_1 = xn + 1$ where $g * x \text{ mod } n^2$ is equivalent to $1 \text{ mod } n^2$. By reversing the process we did in the closure property, for these to multiplied elements we have $gxn^2 + xn + gn + 1 \text{ mod } n^2 = 1 \text{ mod } n^2$. Subtracting 1 to the other side we get $gxn^2 + xn + gn \text{ mod } n^2 = 0 \text{ mod } n^2$ and modulo the n^2 element we get $xn + gn \text{ mod } n^2 = 0 \text{ mod } n^2$. Subtracting xn to the other side we get $gn \text{ mod } n^2 = -xn \text{ mod } n^2$ and dividing by n on both sides we get $g \text{ mod } n^2 = -x \text{ mod } n^2$. Therefore we have $g = -x$ which shows that for every $g \in G_n$, there exists an inverse such that $g = an + 1$ and $g^{-1} = -an + 1$ and taking the mod of these the property is satisfied.

Associativity:

We know this holds true because for any $g = an + 1$, $h = bn + 1$, $k = cn + 1$, we have $g * (h * k) = (an + 1) * (bcn^2 + bn + cn + 1) = abcn^3 + abn + acn + an + bn + cn + 1$. We also have $(g * h) * k = (abn^2 + an + bn + 1) * (cn + 1) = abcn^3 + acn + abn + an + bn + cn + 1$ which is the same output. Therefore for all $g, h, k \in G_n$, this property holds.

Since all properties hold, G_n is a multiplicative subgroup of $\mathbb{Z}_{n^2}^*$.

Exercise 3

(20 points) Consider the Computational Diffie-Hellman assumption (CDH):

Definition 1 (Computational Diffie-Hellman (CDH)). Consider the following experiment

Experiment $\text{CDH}^{\mathcal{A}, \mathcal{G}}(1^n)$.

- Run $\mathcal{G}(1^n)$ to obtain $(\mathbb{G}, q, g)$, where $\mathbb{G}$ is a cyclic group of prime order q and g is a generator of $\mathbb{G}$.
- Let $a, b \xleftarrow{\$} \mathbb{Z}_q$
- $u \leftarrow \mathcal{A}(\mathbb{G}, q, g, g^a, g^b)$
- Output 1 if $u = g^{ab}$.

We say that a p.p.t. adversary $\mathcal{A}$ breaks the CDH if

$$\Pr[\text{CDH}^{\mathcal{A}, \mathcal{G}}(1^n) = 1] \geq \frac{1}{p(n)}$$

for some polynomial $p(\cdot)$.

Also recall the Discrete Logarithm Assumption (DLOG) and the Decisional Diffie-Hellman Assumption (DDH).

Definition 2 (Discrete Logarithm Assumption (DLOG)). Consider the following experiment

Experiment $\text{DLOG}^{\mathcal{A}, \mathcal{G}}(1^n)$.

- Run $\mathcal{G}(1^n)$ to obtain $(\mathbb{G}, q, g)$, where $\mathbb{G}$ is a cyclic group of prime order q and g is a generator of $\mathbb{G}$.
- Let $h \xleftarrow{\$} \mathbb{G}$
- Let $x \leftarrow \mathcal{A}(\mathbb{G}, q, g, h)$
- Output 1 if $g^x = h$.

We say that a p.p.t. adversary $\mathcal{A}$ breaks the discrete logarithm problem if

$$\Pr[\text{DLOG}^{\mathcal{G}, \mathcal{A}}(1^n) = 1] \geq \frac{1}{p(n)}.$$

for some polynomial $p(\cdot)$.

Definition 3 (Decisional Diffie-Hellman (DDH)). Consider the following experiment

Experiment $\text{DDH}_b^{\mathcal{A}, \mathcal{G}}(1^n)$.

- Run $\mathcal{G}(1^n)$ to obtain $(\mathbb{G}, q, g)$, where $\mathbb{G}$ is a cyclic group of prime order q and g is a generator of $\mathbb{G}$.
- Let $x, y, z \xleftarrow{\$} \mathbb{Z}_q$
- If $b = 0$, $\mathcal{A}$ is given $(\mathbb{G}, q, g, g^x, g^y, g^z)$, else $\mathcal{A}$ is given $(\mathbb{G}, q, g, g^x, g^y, g^{xy})$
- Output $\mathcal{A}$'s output.

We say that a p.p.t. adversary $\mathcal{A}$ breaks the DDH problem if

$$|\Pr[\text{DDH}_0^{\mathcal{G}, \mathcal{A}}(1^n) = 1] - \Pr[\text{DDH}_1^{\mathcal{G}, \mathcal{A}}(1^n) = 1]| \geq \frac{1}{p(n)}.$$

for some polynomial $p(\cdot)$.

1. Show that if there exists a p.p.t. adversary $\mathcal{A}$ that breaks the DLOG problem, then there exists a p.p.t. adversary $\mathcal{B}$ that breaks the CDH problem.
2. Show that if there exists a p.p.t. adversary $\mathcal{B}$ that breaks the CDH problem, then there exists a p.p.t. adversary $\mathcal{C}$ that breaks the DDH problem.

Solution

1) If there exists a PPT adversary $\mathcal{A}$ that breaks the DLOG problem, then we can construction a PPT adversary $\mathcal{B}$ that breaks the CDH problem.

We can do this in the following way:

We know that adversary $\mathcal{A}$ breaks DLOG which means it has greater than $\frac{1}{p(n)}$ probability of outputting 1 meaning that given $(\mathbb{G}, q, g, h)$ from the DLOG experiment it outputs $x \in \mathbb{Z}_q$ and wins the game if $h = g^x$. We construct $\mathcal{B}$ using $\mathcal{A}$ to query the information in the CDH game.

We construct $\mathcal{B}$ as follows:

From the CDH game we run the generator and uniform random generator to get an output of $(\mathbb{G}, q, g, g^a, g^b)$. Then $\mathcal{B}$ queries $\mathcal{A}$ on $(\mathbb{G}, q, g, g^a)$. Because of the assumption of $\mathcal{A}$, we know that $\mathcal{A}$ will output an $a' \in \mathbb{Z}_q$ that is the correct random a for g . Then $\mathcal{B}$ uses this output to compute $(g^b)^{a'}$.

If $\mathcal{A}$ succeeds in this game then we know that $\mathcal{B}$ will also succeed. Therefore if we know that $\mathcal{A}$ succeeds with greater than $\frac{1}{p(n)}$ probability for some polynomial p , then it must also hold that $\mathcal{B}$, while succeeds every time $\mathcal{A}$ succeeds under our construction, also has a success of greater than $\frac{1}{p(n)}$ probability for some polynomial p of outputting 1. From the definition in the problem this probability for the CDH game proves that $\mathcal{B}$ is a PPT adversary that breaks CDH.

2) If there exists a PPT adversary $\mathcal{B}$ that breaks the CDH problem, then we can construct a PPT adversary $\mathcal{C}$ that breaks the DDH problem.

We can do this in the following way:

We know that $\mathcal{B}$ breaks CDH which means that it has greater than $\frac{1}{p(n)}$ probability for some polynomial p of outputting 1 meaning that given $(\mathbb{G}, q, g, g^a, g^b)$ in the game, $\mathcal{B}$ is able to output a $u \in G$ that is equal to g^{ab} . We construct $\mathcal{C}$ using $\mathcal{A}$ to query the information in the DDH game.

We construct $\mathcal{C}$ as follows:

From the DDH game we run the generator to obtain (G, q, g) and uniformly randomly choose $x, y, z \in \mathbb{Z}_q$. Now $\mathcal{C}$ has access to a DDH instance of $(\mathbb{G}, q, g, g^x, g^y, hi)$ where $hi = g^z$ or $hi = g^{xy}$.

Now $\mathcal{C}$ queries $\mathcal{B}$ on $(\mathbb{G}, q, g, g^x, g^y)$ like the format of the CDH game to get a output of hi' from $\mathcal{B}$. If $hi' = hi$ then $\mathcal{C}$ outputs 1, else outputs 0.

Since $\mathcal{B}$ computes hi' using x and y , we have that the probability of $\mathcal{C}$ outputting $b = 1$ if the DDH game gave $\mathcal{C} (\mathbb{G}, q, g, g^x, g^y, g^{xy})$ is greater than $\frac{1}{p(n)}$ because the probability of $\mathcal{B}$ outputting the correct $hi' = g^{xy}$ is greater than $\frac{1}{p(n)}$ due to our assumption.

We also have that if the DDH game gave $\mathcal{C} (\mathbb{G}, q, g, g^x, g^y, g^z)$, the probability of $\mathcal{C}$ outputting 1 is $\frac{1}{q}$ because there is a $\frac{1}{q}$ probability that $z \in \mathbb{Z}_q$ is equal to $g^{xy} \in G$ because q is the order of the cyclic group.

Therefore we get the final probability to be:

$\Pr[\text{output 0 if } hi = g^z] - \Pr[\text{output 1 if } hi = g^{xy}] \leq \text{negl}(n)$. From this we know that q is n in this problem so this side of the probability is also negligible. Thus it holds that the probability is greater than or equal to the $\Pr[\text{output 1 if } hi = g^{xy}]$ which is $\frac{1}{p(n)}$ for some polynomial p for $\mathcal{C}$.

By definition in the problem we have proven that since we got this probability, it must be that $\mathcal{C}$ is an adversary that breaks the DDH problem.

Exercise 4

(25 points) Alice and Bob want to securely compute the following circuit using a garbled circuit scheme. In the following circuit, The input wires marked A and C are Alice's input, and the input wires marked B and D are Bob's input. Alice will act as the garbler, and Bob will act as the evaluator.

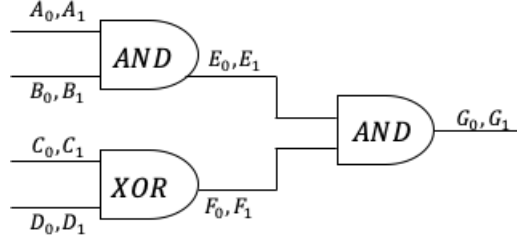


Figure 1: The circuit and the garbled labels on each wire.

Alice does not want to make computational assumptions. Therefore, she decides to encrypt the truth table for each gate using a variant of one-time pad instead. Specifically for the circuit above, she randomly picks label for each wire as follows:

- $G_0, G_1 \xleftarrow{\$} \{0, 1\}^n$.
- $E_b, F_b \xleftarrow{\$} \{0, 1\}^{2n}$ for $b \in \{0, 1\}$.
- $A_b, B_b, C_b, D_b \xleftarrow{\$} \{0, 1\}^{3n}$ for $b \in \{0, 1\}$.

Then to encrypt each entry in the truth table, Alice uses the following scheme.

- $\text{Enc}_{k,k'}(m)$: output $c = k \oplus k' \oplus (0^n \| m)$.
- $\text{Dec}_{k,k'}(c)$: compute $m_0 \| m_1 = c \oplus k \oplus k'$ where $m_0 = |n|$. If $m_0 = 0^n$, output m_1 ; otherwise, output $\perp$.

For example, for the first AND gate, Alice computes

$\text{Enc}_{A_0, B_1}(E_0)$
$\text{Enc}_{A_1, B_0}(E_0)$
$\text{Enc}_{A_0, B_0}(E_0)$
$\text{Enc}_{A_1, B_1}(E_1)$

Unfortunately, this encryption makes the garbled circuit insecure and leaks additional information beyond the inherent leakage (i.e., what is already leaked by the output). In this question, you are asked to show that Bob can break the security of the scheme step by step guided by our hints.

Suppose Bob's input is 0 to wire B and 1 to wire D . We do not know Alice's input to wire A and to wire C . Note that Bob only gets the garbled circuit, garbled input, and the output wire labels.

- Describe how Bob can compute both E_0 and E_1 given the garbled truth tables.
- Suppose Alice's input for wire C is $c \in \{0, 1\}$. Let $\theta = 1 \oplus c$. By the protocol, Bob should be able to learn F_θ . Describe how Bob can tell whether $F_\theta = F_0$ or $F_\theta = F_1$.

- (c) Describe how Bob can compute Alice's input for wire C.
- (d) Show that if Bob's input is 0 on wire B and 1 on wire D, then Bob should not learn c , i.e., Alice's input on wire C, in a secure 2-party computation scheme.

Solution

a) Bob knows what B_0 is and can compute the two encryptions with B_0 and the $A_{0,1}$ values to try the decryption to both encryptions. He then compares the two decryptions and sees which one correctly computes E_0 with the 0^n string in it as expected. Bob knows B has been input as 0 and D has been input as 1 so Bob try all possible combinations of his inputs on the truth tables and can guess which combination of input labels associated with the wires corresponds to which output label. Therefore to get E_1 Bob can examine the output G and the output F and attempt encryption based on the same value for A and a different value for B using the encryption scheme he knows. b) From the XOR for θ we know that if Alice inputs c as 0 then θ will be equal to 1 because $0 \text{ XOR } 1$ is 1. Similarly if Alice inputs c as 1 then θ will be equal to 0 by the XOR with 1 because $1 \text{ XOR } 1$ is 0. Therefore Bob can get F_θ by identifying θ and getting the corresponding output from the gate. After getting F_θ Bob knows his own inputs of b and d he can know that since $D = 1$, the XOR between C and D will be an obvious indicator of what the value of C was. Therefore by knowing this Bob can differentiate between the outputs of F_0 and F_1 . Since Bob knows the encryption scheme Enc and has access to the garbled entries in the truth table Bob can attempt to decipher both F_1 and F_0 by checking the corresponding encrypted outputs based on the combination of input labels he can try. Then Bob knows which output to expect for F_1 and F_0 through the XOR statements of the encryption. By knowing theta Bob can attempt each value for the input label on the Encryption and see whether the output corresponds to what he expects for F_0 or what he expects for F_1 . c) Bob knows that $\theta = 1 \oplus c$ and knows F_θ from part b so c can be gotten by deducing the value of θ . If $F_\theta = F_0$ then $\theta = 0$ and $c = 1$ and if $F_\theta = F_1$ then $\theta = 1$ and $c = 0$. From the XOR gate it is simple to see the value of c from the previous information. d) Under a secure 2-party computation scheme, Bob should not have enough information to recover any inputs that are not his own in the circuit. To show this in a secure computation scheme there can be a circuit like the one given in the problem. We can have the garbling process as such: Alice generates random bit strings for the inputs such that it is the same random input as it was in the problem. For the truth table we have a similar outline. The key difference is in decryption. Bob only decrypts according to his own input either B or D. Both only should know the labels associated with $B = 0$, that is Bob only sees the garbled entries $Enc_{0,0}(E_0)$ or $Enc_{0,0}(E_1)$. Therefore when Bob decodes the garbled entries for A he gets the value of $F_{B=0,A=0}$ without knowing the actual values of A or C directly. This must be secure because the method obscures E_0 and E_1 . Even if Bob learns the output for D he cannot derive C because the output for

the circuit depends on A as well as C . If Bob tries to derive Alice's input as $C=0$ Bob cannot because decryption only relies on his input and cannot differentiate between the value of A or the value of C being 0 if the output is 0, leaving him with a $1/2$ probability only. A and C are also random so there is no way to receive the same random output if Bob were to try and compute the entire process again, therefore the difference between the strings and bits cannot be determined.

Exercise 5

(20 points) Alice and Bob want to securely compute the following circuit using a garbled circuit scheme. In the following circuit, The input wires marked A and C are Alice's inputs, and the input wires marked B and D are Bob's inputs. Alice will act as the garbler, and Bob will act as the evaluator.

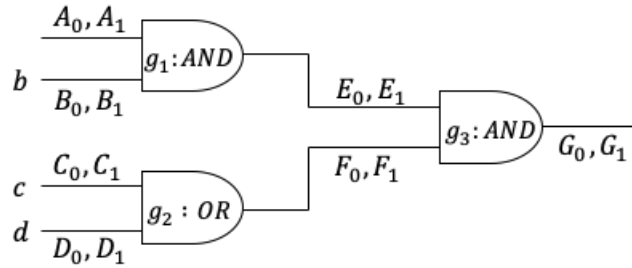


Figure 2: The circuit and the garbled labels on each wire.

Let $\hat{g}_1, \hat{g}_2, \hat{g}_3$ be the garbled gates of g_1, g_2 and g_3 , respectively. The input labels are plotted in the figure. In this problem, we assume that all labels are of length n . Now suppose that an adversary Charlie learns the circuit, the garbled gates $\hat{g}_1, \hat{g}_2$ and $\hat{g}_3$, output wire labels G_0, G_1 , as well as input wire labels A_0, A_1, B_b, C_c, D_d , where $b, c, d \in \{0, 1\}$ are the inputs to the input wire labeled B, C and D , respectively. Show how Charlie can use this information to learn at least one input out of b, c , or d with probability $1 - \text{negl}(n)$ for some negligible function negl . (Hint: How should the output of an AND gate change when you change one of the inputs? What does this tell you about the other input?)

Solution

We know that A, C are Alice's inputs and Bob's inputs are B and D . The adversary knows the circuit, the gates, the output for either G_0, G_1 , and the input wire labels for A_0, A_1, B_b, C_c, D_d , where $b, c, d \in \{0, 1\}$. From this information that Charlie has we can construct an attack where Charlie can derive information about the inputs. For the first AND gate g_1 Charlie knows that this AND gate computes the output

given the A and B inputs. Since Charlie knows the output, he can infer Bob's input B because changing B from B_0 to B_1 the output of g_1 will change to either 0 or 1 depending on Alice's input A. Since A is not known, changing the input (for example if g_1 changes from 0 to 1 then it is inferred that $A = 1$ and $B = 1$). If it stays the same Charlie can eliminate that scenario.

For the gate g_2 we have an OR gate between inputs C and D. Charlie can change D by switching between D_0 and D_1 to see the output of g_2 change. For example if the output of g_2 changes from 0 to 1 then he can infer that $C = 1$ or $D = 1$ and if it stays the same he knows for sure that $C = 0$ and $D = 0$.

For the last gate Charlie can simply check the outputs of g_1 and g_2 and learn the output of g_3 and confirm that it is the same. This reveals more information about A and B and C and D from the two previous gates.

There are different cases for where Charlie can learn the inputs.

If for the first gate, Charlie changes B and the gate produces 1, Charlie knows that A and B in that scenario are 1 and therefore knows the state of A and B. Then Charlie knows input value b .

For the other case if Charlie changes D and the output of the second gate changes from 1 to 0, Charlie knows that both inputs C and D are 0 and therefore knows the input of c, d . If it goes from 1 to 0 and 0 to 1 when Charlie changes D to 0 then 1 Charlie knows that C is at input 0 and can infer d with high probability.

Done in the same way Charlie can infer c .

Probability analysis:

Charlie's probability of deducing b from gate 1: if the output is 1 there is a probability of 1 for knowing bit b because the two inputs must be 1 at the switch. If the output stays at 0 then Charlie has a $\frac{1}{2}$ probability of getting b correct since he does not exactly know what A is.

Deducing c and d from gate 2: Charlie knows that if the output is zero then the two inputs must be zero and has a probability of 1 for outputting c, d correctly. Charlie has a $\frac{1}{2}$ chance of deducing d correctly if there is an effect on the output after switching D. For gate three: We know that if both E and F are 1 then only G can be 1 and then we know A and B must be 1 too, so b is always known here. Otherwise we have $\frac{1}{2}$ probability of deducing b or d .

In the case with the least information given, we have output of 0 for the AND gate and output 1 for OR gate. Since A is always constant, we know that if A is 1, changing B from 0 to 1 to 0 will never change the output to 1. If A is 0 we will never know b . For the OR gate in the scenario A is 0 we know g_3 will output 0 as well, and we can change inputs C and D many times until we notice a pattern of outputs, then we can deduce either c or d .

Therefore in any scenario we always have a high probability ($1 - \text{negl}(n)$) of getting at least one bit's true value.